MACHINE LEARNING FOUNDATIONS

Linear Regression

Core Theory, Mechanics, Assumptions & Real-World Applications

Why We Need Linear Regression

Without it, modeling real-world quantitative relationships means complex algorithms or arbitrary guesswork

Quantitative Forecasting

Business, engineering, and science constantly need continuous forecasts - next month's revenue, a house value, fuel use. Linear Regression gives a mathematically proven way to model the trend and compute exact outputs from inputs.

High Interpretability

Black-box models like deep nets or random forests predict well but can't explain why. Linear Regression outputs explicit weights: a weight of +50,000 for "years of experience" tells stakeholders every extra year adds a fixed amount to salary.

Baseline Benchmarking

Jumping straight to a complex model makes it hard to know if accuracy comes from real signal or over-engineering. Linear Regression is the fast, standard baseline - if a complex model can't beat it, the added complexity isn't earning its keep.

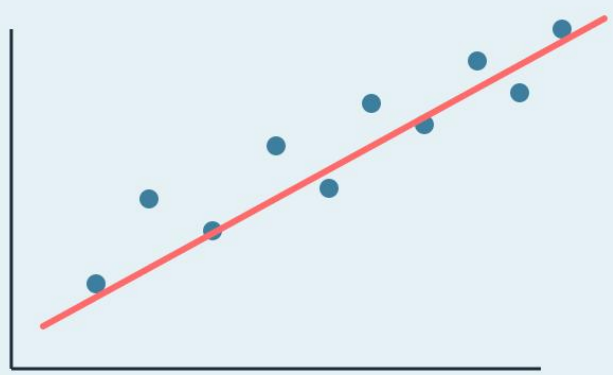
Low Compute Requirements

Deep learning demands heavy GPU power, energy, and memory. Linear Regression trains in milliseconds via simple matrix operations - ideal for edge devices, real-time microservices, and streaming data.

What Is Linear Regression?

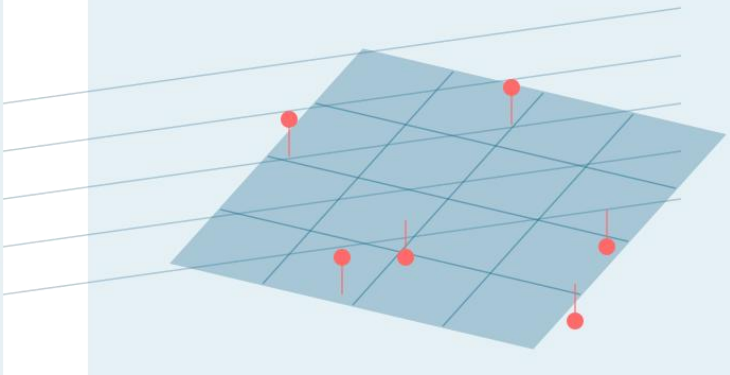
A fundamental **parametric, supervised** machine learning algorithm that predicts a **continuous numerical output** from one or more input features, assuming an additive, linear relationship between them.

y = mx + c



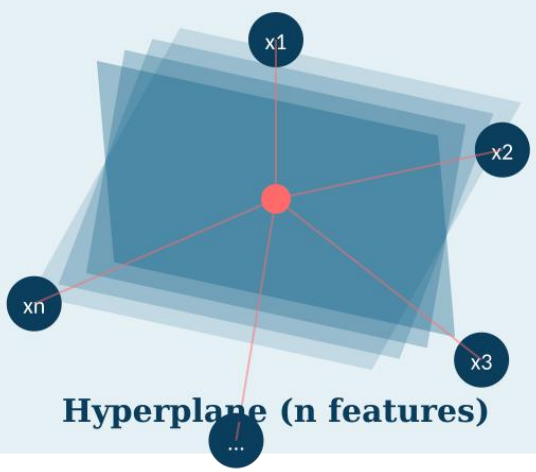
Straight Line (1 feature)

y = c + m1x1 + m2x2



Flat Plane (2 features)

y = c + Σ mi xi



Hyperplane (n features)

Geometrically: a line in 1 dimension, a plane in 2 dimensions, and a hyperplane beyond that.

TYPE

Supervised · Parametric

OUTPUT

Continuous (regression)

CORE IDEA

Best-fit straight line / plane

Mathematical Formulation

Simple Linear Regression

1 input feature

$$y = mx + c + \varepsilon$$

m slope — change in y per 1-unit rise in x

c intercept — baseline value of y when x = 0

ε irreducible random error / noise

Multiple Linear Regression

n input features

$$y = c + m1x1 + m2x2 + \ldots + mnxn + \varepsilon$$

Vector form: $\hat{y} = XW^T + c$

W = [m1, m2, …, mn] is the weight vector

X is the feature matrix

y — Target

The value being predicted, e.g. salary or price

x — Feature

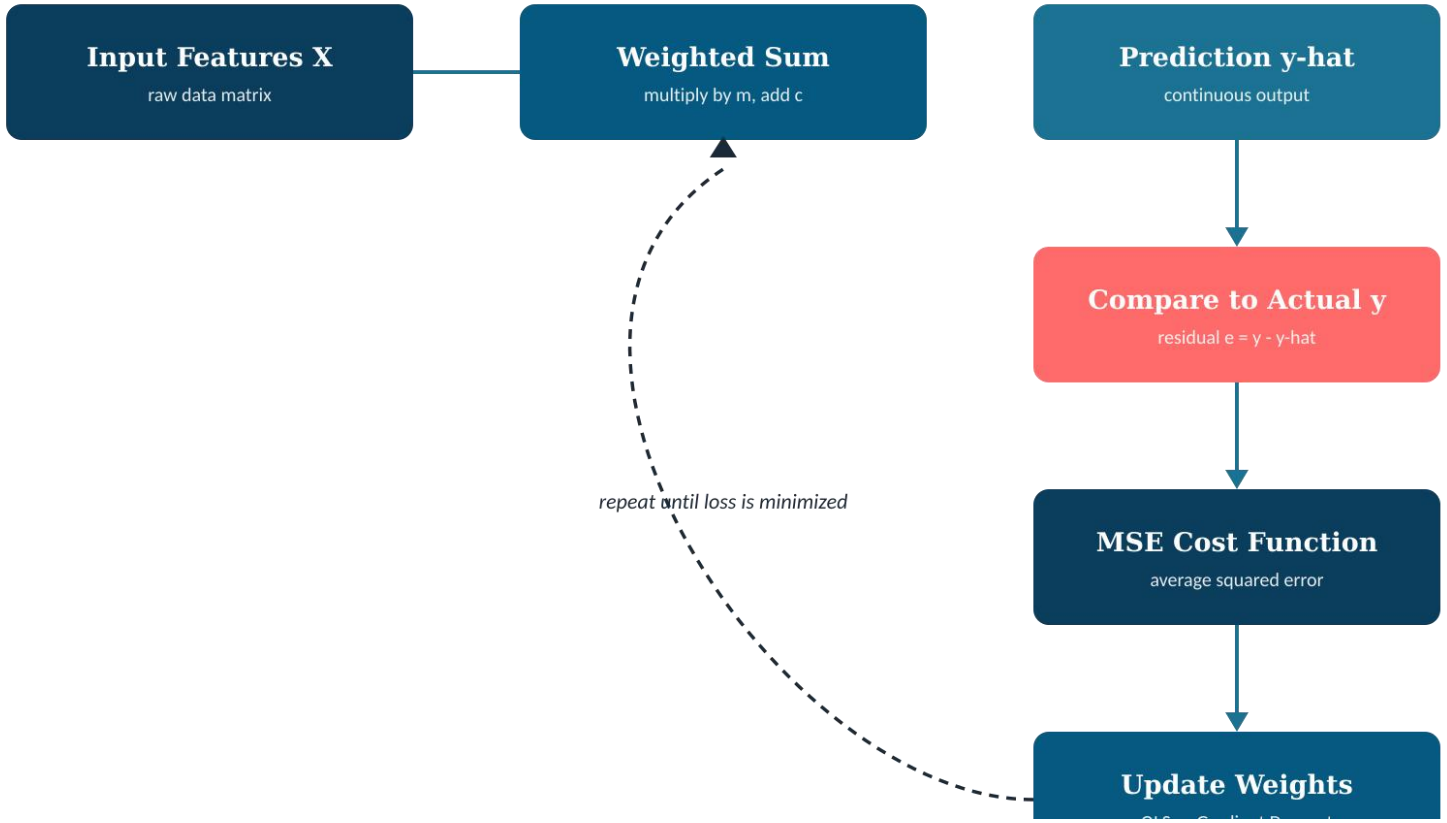
The input used to make the prediction

m — Weight

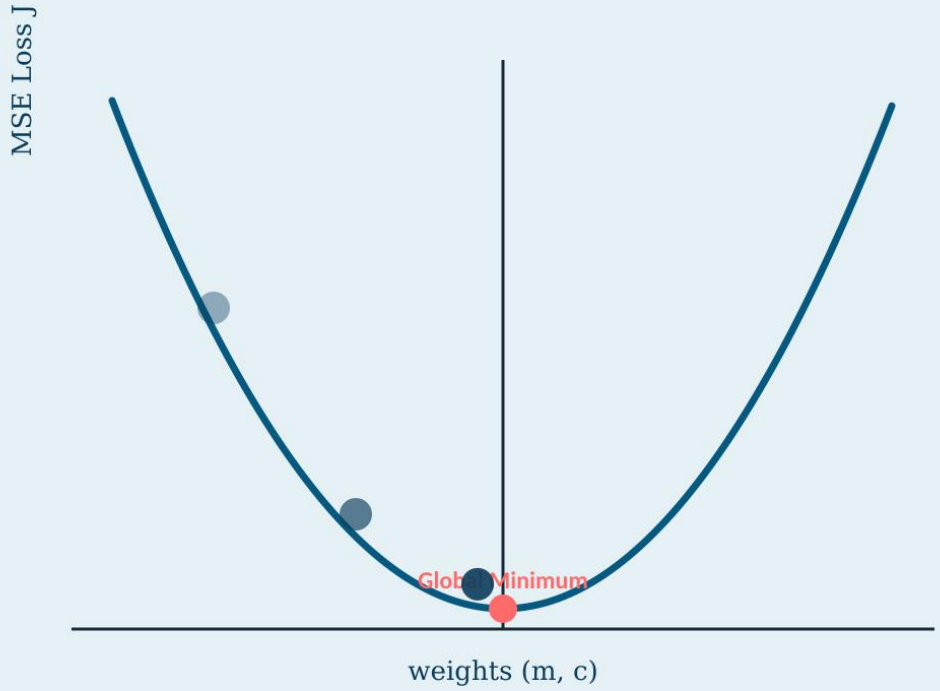
Learned coefficient controlling each feature's influence

How It Works: The Training Pipeline

From raw features to a tuned prediction line



The Cost Function: Mean Squared Error



$$J(m, c) = (1/N) \Sigma (yi - \hat{y}i)^2$$

The residual e = y − ŷ measures the gap between actual and predicted values. Squaring it gives a smooth, convex "bowl" with one guaranteed global minimum.

Cancels sign conflicts

Positive and negative errors no longer cancel each other out

Penalizes big misses

Larger errors are punished far more than small ones

Optimizable by calculus

A smooth, differentiable surface with one global minimum

Optimization: Learning the Weights

Ordinary Least Squares (OLS)

$$W = (X^T X)^{-1} X^T Y$$

✓ Exact, closed-form solution — one step, no learning rate

✗ Matrix inversion cost grows ~O(n³) — expensive past ~100K features

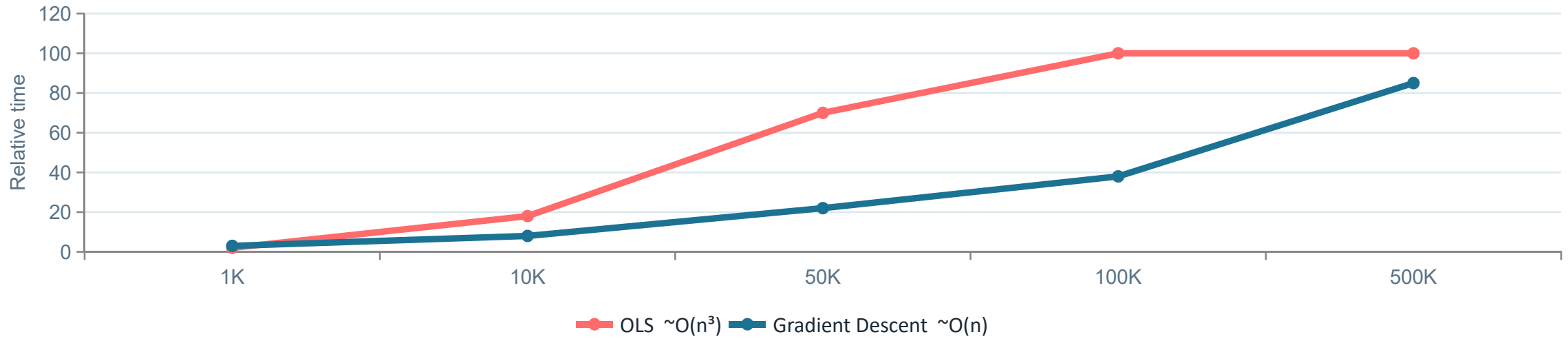
Gradient Descent (Iterative)

$$mj := mj - \alpha \cdot \partial J / \partial mj$$

✓ Scales to millions of rows and features — ~O(n) per step

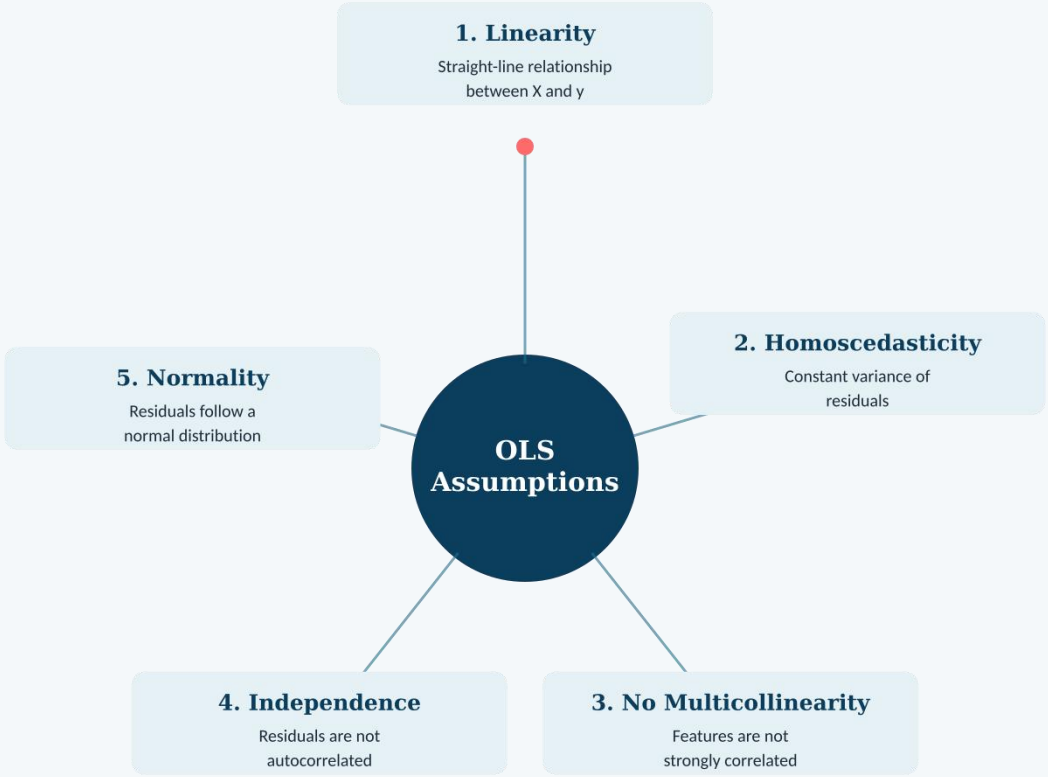
✗ Needs a tuned learning rate α and multiple iterations

Illustrative Training-Time Growth as Feature Count Increases



Five Classical Assumptions of OLS

Required for unbiased, minimum-variance estimates (Gauss-Markov Theorem)



Strengths & Limitations

STRENGTHS

Highly Interpretable

Every weight directly explains a feature's isolated effect on the target

Resists Overfitting

Low model complexity keeps it stable on small datasets

Lightweight & Fast

Cheap to train, evaluate, and deploy — even on edge devices

LIMITATIONS

Only Captures Linear Patterns

Underfits curved or multi-tiered relationships

Sensitive to Outliers

Squared error lets a few extreme points pull the whole line off target

Hurt by Multicollinearity

Correlated features confuse which one truly drives y

Scale-Dependent

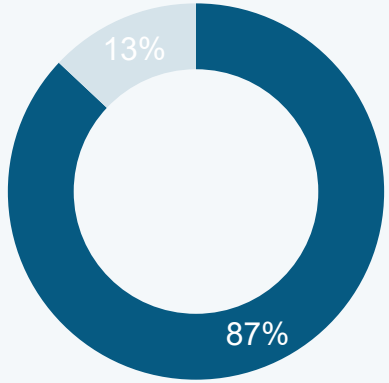
Unscaled features slow or distort Gradient Descent optimization

Model Evaluation Metrics

Regression can't use "accuracy" — continuous distance metrics measure fit instead

Metric	Measures	Key Trait
MAE	Average absolute error, in original units	Easy to interpret · robust to outliers
MSE	Average squared error	Heavily penalizes large mistakes
RMSE	Square root of MSE	Back in original units · still outlier-sensitive
R^2 Score	% of variance in y explained by X	Scale 0.0 – 1.0 (1.0 = perfect fit)
Adjusted R^2	R^2, penalized for irrelevant features	Prevents false inflation from adding noise

R^2 Score — Example Model



Explained by model Unexplained variance

$R^2 = 0.87$

Regularization: Taming the Weights

Standard OLS has zero hyperparameters — penalty terms add control over overfitting

Ridge (L2)

$$\text{MSE} + \alpha \Sigma\, mj^2$$

Shrinks coefficients toward zero — great for multicollinearity

Lasso (L1)

$$\text{MSE} + \alpha \Sigma\, |mj|$$

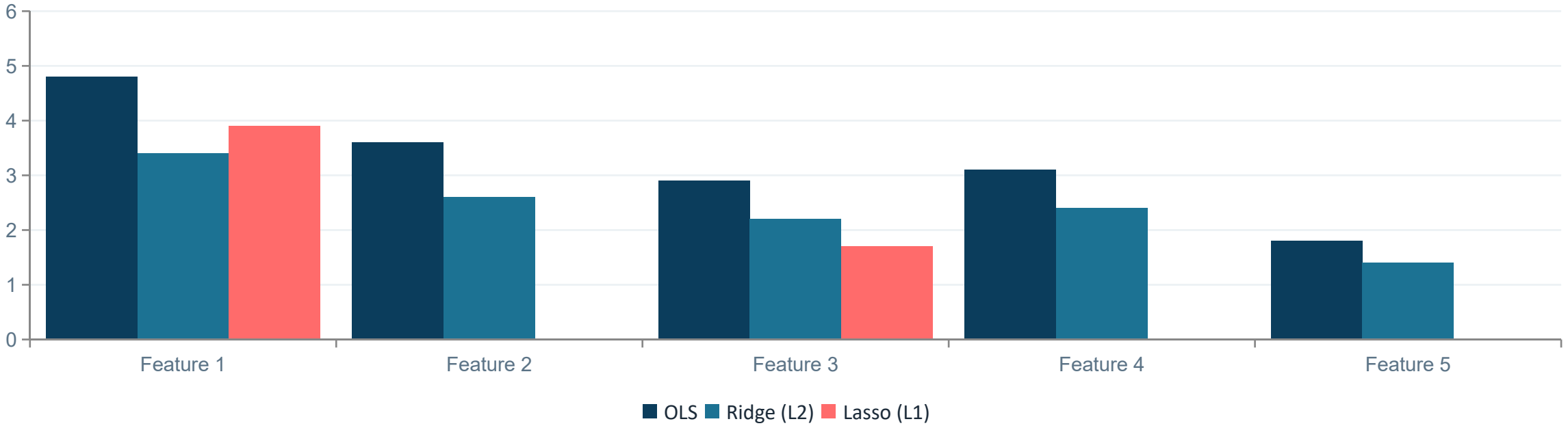
Forces weak coefficients to exactly zero — automatic feature selection

ElasticNet

L1 + L2 combined

Balances both penalties via the l1_ratio parameter

Illustrative Effect: Coefficient Magnitude by Method



Real-World Applications

Real Estate Valuation

Estimating market price from square footage, room count, and neighborhood rating

Sales & Demand Forecasting

Predicting quarterly product demand from ad spend, season, and regional pricing

Financial Risk Analysis

Computing stock beta — asset volatility relative to the broader market

Healthcare & Life Sciences

Modeling baseline blood pressure from age, BMI, and daily sodium intake

Key Takeaways

- Linear Regression fits the straight line, plane, or hyperplane that minimizes squared error
- Weights are learned via OLS (exact, small data) or Gradient Descent (scalable, big data)
- Five classical assumptions must hold for OLS estimates to be statistically valid
- Ridge, Lasso, and ElasticNet regularize the model to fight overfitting and multicollinearity
- Interpretable and lightweight — the natural first baseline for any regression problem

Thank You